ProjectConnect

The Complete Handbook — all content requested on 12 October 2026

Part I · End-to-End Master Prompt — the full build contract

Part II · Tech Stack — every technology, its version, and its use

Part III · End-to-End Flow — the beginner-developer flowchart

Part IV · Full Website Use — for a USER, screen by screen

Part V · Full Website Use — for the OWNER / COMPANY

Part VI · Revenue Streams — pricing mapped to enforcement that exists

Part VII · Launch, Deploy & Hosting Playbook — the recommended method

Part VIII · Sales & Marketing — 90-day plan, channels, buyers, budget

Source of truth: `/home/user/projectconnect` · 118/118 automated tests passing
Status: NOT READY FOR PRODUCTION LAUNCH — remaining gates in docs/LAUNCH_STATUS.md

Part I — Master Prompt

The entire project contract: mission, scope, stack, data invariants, flows, validation gates, runbook, deliverables.

Everything we built, from the first idea to the launch gates. Pasted into any capable coding agent, this prompt reproduces or continues this product with nothing missing.

(Condensed 12 Oct 2026 from the original spec `~/ProjectConnect_Master_Prompt.md`, the launch contract `docs/LAUNCH_CONTINUATION_PROMPT.md`, and every checkpoint since. Superseded narrative removed; all binding requirements kept.)

---

0. Mission and unbreakable rules

You are building **ProjectConnect** — a community platform where makers (students, hobbyists, engineers) find teammates, launch real projects (drones, renewable energy, agriculture tech, hardware), and get things built. Not a prototype. Not an MVP. A launch-ready product; you stop only at gates that require an owner's credentials, money, or legal approval.

1. **Execute, don't plan.** Work autonomously through every reversible engineering decision; no permission-checks between phases. Ask only for: missing credentials, spending, destructive acts on real data, external publication.
2. **No fake done.** A feature is complete when validated inputs, enforced authorization, persisted state, error/loading/empty states, and passing tests exist — not when a button renders. No canned responses, no silent catches, no weakened assertions, no skipping tests to go green.
3. **"No errors" = zero known failing required checks**, with residual risks explicitly documented. Never claim error-free software or a launched product unless an authorized deployment + production smoke actually passed.
4. **Evidence over prose.** Keep real command outputs, totals, and limits in `docs/VALIDATION_REPORT.md`. Label everything `implemented+verified / implemented+unverified / not implemented / blocked-manual`.
5. **Secrets discipline.** No production secrets in code, docs, logs, screenshots, archives. `.env.example` documents names only; seeds never run against production.
6. **Checkpoint always.** After each coherent change: save source + docs to the workspace so any next session can continue cold.
7. **Preserve the design system** (navy/cyan/purple, Manrope, responsive, WCAG-AA) and the working preview while evolving.

1. Product definition

- **Personas:** student builder needing teammates; solo maker seeking skills/parts; mentor/club looking for projects to sponsor; hardware hobbyist finding local parts.
- **Value loop:** showcase a project → attract skilled applicants → vet & accept → collaborate in a workspace → ship → reputation compounds → discover again.
- **Differentiators:** project-centric (not résumé-centric), real collaboration tooling (tasks/files/chat), local hardware discovery, an AI copilot under hard budgets, privacy-by-default public profiles.

2. Full feature scope (A–H) — every one is a hard requirement

A. Identity, accounts, privacy

Email register/login/logout; bcrypt hashing; email verification + forgot/reset via one-use expiring hashed tokens; no account enumeration (identical failure messages); mail through a durable DB outbox with

retry. Google OAuth (server-side exchange, state+nonce+PKCE, JWKS verify, working "not configured" states). Revocable sessions (list + revoke-one/revoke-all), password change revokes sessions, CSRF via exact-origin allowlist, HttpOnly cookies (Secure+Lax; None+Partitioned only behind the sandbox-preview flag). Onboarding: role, skills, interests, education, city, portfolio, avatar. Public serializers expose only explicitly public fields — emails/hashes/token internals never leak through nested payloads. Account data export (ZIP of signed sections) and hard delete with retention-safe dependency handling. Notification preferences, consent states, privacy toggles. Legal pages marked draft until owner/legal approves — never invent approval.

B. Social & discovery

Public profiles and showcases; follow/unfollow; connection request/accept/reject/cancel; block (enforced in messaging AND visibility); reciprocal-request normalization; uniqueness safe under concurrency. Activity feed (own posts, delete), notifications with unread counts, explainable match scoring (skills/interests/city/public activity — fit-v2), search + filters server-side where data is server-held, cursor pagination, signed stateless pagination tokens. Reports → moderation queue. Stable shareable URLs `/projects/[id]`, `/people/[id]`, metadata, robots + partitioned sitemap index; privacy respected in all SEO payloads.

C. Projects & workspaces

Create/edit/archive/delete; cover + gallery images; skills/tags, open positions with capacity, location/remote, status; server search/filter/sort. Apply with message + what-you-bring; cancel/reapply rules; owner review accept/decline; **atomic capacity enforcement**; idempotent repeat-acceptance. Teams: roles, member add/remove, owner transfer, leave, invitations (generate/accept/revoke), audit events per workspace. Workspace UI: Overview, Tasks (Kanban, drag/drop + keyboard alternative, order/assignee/due/labels), Milestones (progress computed from persisted tasks), Files (private, quota'd), GitHub links (public metadata cache; private token isolated + revocable), Updates feed. Chat: per-project channels + direct messages; realtime where available with **documented polling fallback**; bounded gap-safe catch-up, deletion-independent cursors, duplicate-safe explicit retry, unread acks, late-response isolation; blocking enforced; IDOR impossible on every channel/attachment.

D. Resource hub

Real persisted resources (repos, docs, APIs, libraries, tutorials, datasets, tools); create/edit/moderate/search/filter/paginate; bookmarks/upvotes; attach/detach to projects with membership checks. GitHub metadata adapter: single-flight cache, shared quota budget, backoff. **SSRF controls on all server-side fetches** (protocol allow, private-IP block, redirect + DNS-rebinding recheck, timeouts, size caps). No fake data when keys absent — truthful "not configured" states.

E. Nearby hardware & maps

Store records with categories, hours, phone, directions links, favorites; community submissions with verification states and reporting; permission-aware location/radius search with tested distance math (test against known points; never fabricate live results). Google Places Text Search **server-side**; browser Maps display (when ever enabled) uses a domain-restricted key and Google attribution assets; transient (non-stored) list data per Google policy; per-user/day query quotas; graceful no-key/no-location states.

F. AI assistant

Provider interface + real adapter, explicit enablement; per-user/project/global daily budgets in **integer micro-USD with an owner-approved unit price**; ledger rows per call; streaming with backpressure, timeout cancel, error/abort UI; persisted private conversations with consent + deletion; structured-output zod validation; safe rendering (no raw HTML); citations marked verified/unverified — no fabricated repos, stores, or "certified engineering" claims; drone/electrical guidance carries safety disclaimers; roadmap→task import requires confirmation, membership authz, duplicate prevention.

G. Administration

Admin panel: moderation queue (reports, content takedown, reasons), user suspend/restore, audit log, aggregate ops metrics incl. quarantine/export telemetry with alert codes — private file metadata never exposed to non-admins. Least privilege: role checked on every mutation.

H. Polish & operations

Every view has loading/error/empty/offline states; 320px+ responsive, no horizontal overflow; dialogs focus-trapped, Esc-dismissable, visible focus, reduced-motion; AA contrast in-suite; owned/licensed art only (illustrations labeled as concepts, never disguised as project photos). Docker images non-root with healthchecks; compose for dev and deploy; migrations-before-rollout one-shot service; HTTPS via Caddy; secure headers/CSP matched to OAuth/Maps needs; structured redacted logs + request trace IDs; readiness `/api/ready` + liveness `/api/health`; systemd monitor with real alert logic; CI workflow (disposable container); backups + **tested restore**, not just dump scripts.

3. Tech stack (exact, audited)

TypeScript 5.9 everywhere; Node 22 in containers. **Front:** Next.js 16.4 App Router, React 19.3, hand-written CSS (globals + design refresh; no Tailwind), self-hosted `@fontsource-variable/manrope`, lucide-react icons. **Back:** NestJS 11 on Express 5; zod 4 input DTOs; bcryptjs; cookie-parser; jose for OIDC; fflate for export zips. **Data:** Prisma 6 → PostgreSQL 17 (42 models/43 tables, 25 versioned migrations — never db-push in prod); Redis 7 via ioredis for rate limiting; DB-backed `EmailOutbox` for mail. **Files/security:** ClamAV over raw INSTREAM socket (no wrapper dep), `sharp` byte verification, S3-compatible durable storage + short signed URLs, sha256 digests, persistent scan queue + quarantine. **Ship:** Dockerfile (single node:22 image, `npm ci` →builds), deploy compose (postgres/redis/clamav+signatures/migrate/api/web/caddy proxy), systemd monitor, scripts for smoke/verify. **QA:** Playwright as the single test runner (118 tests, API+browser+axe a11y), ESLint+typescript-eslint, Prettier, tsc strict; custom verifiers (`verify-production-smoke.mjs`, `verify-migration-upgrade.py`, `verify-local-recovery.mjs`, `verify-clamd-local.ts`, `storage-rescan.ts`, release packager with sha256). Unused deps get flagged, not hidden.

4. Data & correctness invariants (must hold in every version)

- Sessions keyed by **hashed** tokens; unique constraints on (user, project) applications, connection pairs, one owner-per-project; FKs with restrictive delete where orphan data would leak (export pins, scan rows).
- Capacity, acceptance, milestone progress, unread counts — computed in transactions, tested under duplicate/concurrent submits.
- All lists: bounded page sizes + stable ordering; pagination cursors signed + expiring.
- Uploads: magic-byte verification → temp → scan queue → sha256 → durable rename **never overwriting**; generation+DB-clock fencing on scan state; quarantine removes bytes from serving; audience check on every read path (`/api/images/:id`).
- Migrations: forward-only numbered SQL, fresh-DB checksums **and** data-upgrade tests from previous releases (20→25, 22→25 verified patterns).
- Error envelopes consistent; Prisma errors mapped; internals never serialized; every response through a DTO.

5. End-to-end flows (normative — see `docs/BEGINNER_FLOWCHART.md` for the diagrams)

1. **Arrive:** `GET /` server-renders; `/api/projects` works anonymously; `/api/me` →401 = guest mode.
2. **Register:** validate→hash→inactive user→activation mail (outbox)→verify→active.
3. **Login:** exact-origin CSRF → rate-limit → bcrypt → session row → `pc_session` cookie → guard authenticates all later calls.

4. **Core loop:** browse/search → open → apply → owner reviews → accept → connections → chat → notifications.
5. **Upload:** client type/size check → POST → server byte-check → ClamAV → sha256+durable → refetch → audience-checked GET → PreviewImage (loading/OK/failed+retry).
6. **Every write** passes: CSP → rate limit → origin → session → zod → authz → Prisma → DTO.
7. **Ship:** install → containers → migrate → seed(dev) → build+test → staging env → smoke → backups/rehearsed restore → launch checklist.

6. Validation gates — run, record, fix; never skip

1. Clean `npm ci` from lockfile, generate, clean migrate, seeded dev boot.
2. Lint, strict typecheck, **both** production builds.
3. Full Playwright suite green (currently 118; provider-fixture API on :3004 for integration specs — see INTEGRATIONS.md).
4. Unit-level: validation, distance math, token/session logic, authz matrix, rate limits.
5. Browser flows: register/verify/recover, login/logout, profile edit, create/apply/accept, chat, workspace files, notifications, settings, unconfigured-integration states — self-cleaning fixtures, no shared demo mutations.
6. Adversarial: IDOR, CSRF/origin, blocked-user messaging, malicious uploads/URLs, XSS strings, quota abuse, stale/revoked sessions, enumeration.
7. Accessibility: automated scan + keyboard smoke; no serious/critical unreviewed findings; mobile overflow check.
8. Dependency audit + secret scan + license inventory, triaged honestly (dev-only advisory ≠ production risk statement).
9. Production-mode smoke behind the real proxy path; health/readiness; restore exercise; bounded load test recorded as sample, not certification.
10. Live third-party tests only with real credentials + explicit authorization; otherwise mark blocked and continue elsewhere with fixtures.

7. Runbook

Fresh: `npm ci` → root `docker-compose up -d` (pg/redis) → `.env` from `.env.example` (DATABASE_URL, REDIS_URL, APP_ORIGINS exact, PUBLIC_APP_URL, API_PORT, SMTP_, MAIL_FROM, GOOGLE_CLIENT_ID/SECRET, GOOGLE_PLACES_, OPENAI_*, CLAMD_PORT, AI budget keys) → `npm run db:generate && npm run db:migrate && npm run db:seed` (dev) → `npm run start:api` then `npm run start:web` (web proxies INTERNAL_API_URL). Sandbox preview additionally: `PC_SANDBOX_PREVIEW=true` at build+run and the exact `https://3000-<sandboxid>.e2b.app` origin (auto-added in dev). Backend is not watch-mode: rebuild `npm run build:api` + restart after edits. Deploy: build image → compose up (migrate runs first) → Caddy TLS → monitor timer → smoke via `verify-production-smoke.mjs`.

8. Manual, owner-only prerequisites (create `docs/MANUAL_SETUP.md`, then stop there)

Domain/DNS/hosting + billing; production Postgres/S3/SMTP credentials; Google Cloud project: OAuth consent screen + authorized redirect URIs, Maps/Places activation with billing caps and key restrictions; GitHub App/OAuth registration; LLM vendor account + approved model & unit price; monitoring/alert accounts; legal approval of policy drafts; final publication authorization. Never fabricate any of these; build adapters + fixtures around them and keep going.

9. Current status & what remains (as of 12 Oct 2026 — do not re-do, do not overclaim)

Decision: NOT READY FOR PRODUCTION LAUNCH. Implemented+locally-verified: everything in §2 A–H (118/118 tests, 25-migration upgrades incl. legacy data, 43-table encrypted restore drill, real local ClamAV EICAR, taste-skill redesign, preview-origin/cookie + image-preview fixes). Agent-owned remainder: full typed DTO/OpenAPI payload parity; patched/pinned scanner + freshness monitoring + adversarial corpus; live S3 immutability/pin/retirement tests; process-kill & large-volume export recovery; multi-replica race and representative-load evidence beyond small samples; statistical enumeration/abuse review; manual assistive-tech + independent security; model/citation/retention quality; executed Docker/hosted CI/Caddy/edge/external alerts/tracing/off-site RPO-RTO. These are tracked in `docs/LAUNCH_STATUS.md`; owner prerequisites in §8 stay owner-only.

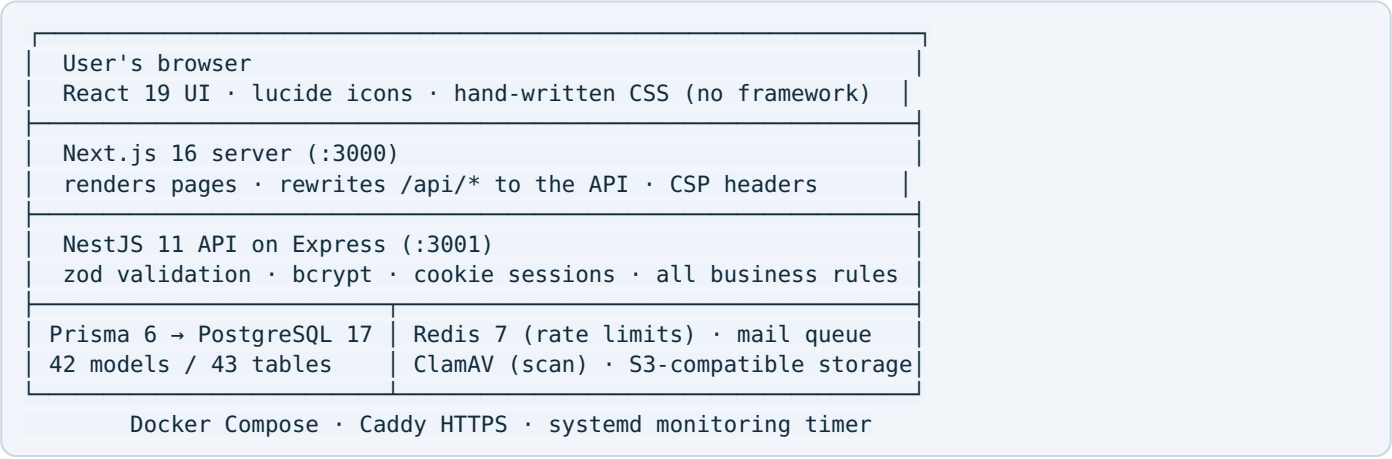
10. Deliverables at every finish line (workspace, always)

Editable repo `/home/user/projectconnect`; `docs/`: MASTER_PROMPT (this file), BEGINNER_FLOWCHART.md/.html, TECH_STACK.md, USER_GUIDE.md, OWNER_GUIDE.md, REVENUE_STREAMS.md, LAUNCH_PLAYBOOK.md, SALES_MARKETING.md, LAUNCH_STATUS, VALIDATION_REPORT, ARCHITECTURE, API, SECURITY, OPERATIONS, INTEGRATIONS, MANUAL_SETUP, checkpoint docs, evidence logs; README + `.env.example`; `LAUNCH_HANDOFF.md` as the human entry point; sanitized release archive + sha256 at the workspace root (no secrets/builds/deps/personal data); the combined PDF handbook built by `scripts/build-handbook-pdf.py`. State plainly: NOT READY vs LAUNCH CANDIDATE, with the failing/unfinished gates listed. No autonomous background work may be implied unless a tracked process actually runs.

Part II — Tech Stack

Audited versions from package.json, Dockerfile, deploy/ and source — with what each is used for.

> Companion to `BEGINNER_FLOWCHART.md`. Every version below was read from `package.json`, `docker-compose.yml`, `Dockerfile`, `deploy/` and the source itself — not from memory. (Audited 12 Oct 2026.)



1 · Languages & runtime

Tech	Version used	What it does here
TypeScript	5.9.3	One language everywhere — web, API, tests, Prisma client. Whole repo passes `tsc --noEmit`.
Node.js	22 in the Docker image (`node:22-bookworm-slim`); dev sandbox runs 20	Runs the API, the build pipeline and the scripts.
tsx	4.23.15	Runs TypeScript directly for dev (`npm run api`) and scripts (`db:seed`, `storage:rescan`).

2 · Front end (`apps/web`)

Tech	Version	Use
Next.js	16.4.0	Server-renders the app shell; its `rewrites()` proxies `/api/:path*` to `INTERNAL_API_URL`, so the browser only ever talks to one origin (that's what lets cookies work same-site). Also serves CSP headers.
React	19.3	UI components — `page.tsx` plus the shared components (`ImageUpload`, `PreviewImage`, `Workspace`, `AiAssistant`, …).
CSS (hand-written)	globals.css + design-refresh.css	Styling, layout and the taste-skill redesign. **No Tailwind / no component library** — deliberate: fewer build layers, full control of contrast/accessibility.
@fontsource-variable/manrope	5.3.0	Self-hosted variable font (works offline, no Google-fonts fetch at page load).
lucide-react	1.53	Tiny SVG icons (sparkles, x, check…).

3 · Back end (`apps/api`)

Tech	Version	Use
NestJS (`@nestjs/core`, `common`, `platform-express`)	11.2.7	

Tech	Version	Use
		Structure for the API: controllers per feature (`main.ts`, `features/*.ts`), dependency injection, exception layer for clean 4xx messages.
Express (via platform-express)	5	HTTP layer underneath; cookie reading, static image streaming, SSE (event-stream).
zod	4.6.5	Validates **every** request body (register, apply, message, upload JSON…) before touching the DB; also types the DTOs.
bcryptjs	3.0.3	Password hashing for accounts and the seeded demo user. Never stored in plain text.
cookie-parser	1.4.7	Reads `pc_session` (login) and `pc_oauth` (state/nonce/PKCE binding) cookies.

4 · Data layer

Tech	Version	Use
Prisma	6.12	Single source of truth for the schema (`prisma/schema.prisma`, **42 models / 43 tables**), type-safe queries, 25 versioned migrations applied with `prisma migrate deploy`.
PostgreSQL	17-alpine (Docker)	All persistent data: users, projects, applications, messages, sessions, notifications, files, scans, audit.
Redis	7-alpine (`ioredis` 6.0 via `REDIS_URL`)	Rate limiting / abuse counters (e.g. login attempts) so a flood can't hammer Postgres.
fflate	0.8.3	Streams zip archives for the account-data export jobs — no heavyweight dependency.

5 · Security pieces

Mechanism	Where	Use
Session cookies	`core/security.ts`, `core/preview-config.ts`	HttpOnly, path=/, Secure in prod, SameSite=Lax (None+Partitioned only for the sandbox preview); hashed tokens in DB, revocable per device.
Exact-origin CSRF check	`core/security.ts`	Every write must come from an allow-listed origin → the 403 `ORIGIN_FORBIDDEN` guard.
CSP & headers	`apps/web/next.config.ts`, `deploy/Caddyfile`	`frame-ancestors`, `nosniff`, HSTS, `X-Frame-Options DENY`, referrer policy.
jose (JWT/JWKS)	`core/oidc.ts`	Verifies Google's ID tokens with remote JWKS; OAuth uses state + nonce + **PKCE**.
Rate limiting	`core/rate-limit.ts` + Redis	Per-IP buckets on auth and write routes.
Input validation	zod everywhere	Unknown fields (like `role`, `status`) can never be forged through DTOs.

6 · Files & external services

Tech	Use
ClamAV (own client, no npm dep)	`core/file-scanner.ts` speaks the raw `INSTREAM` socket protocol — every upload scanned before it becomes durable.

Tech	Use
sharp 0.35	Decodes/inspects real image bytes (dimensions, format) server-side in `files.ts` / `public-image.ts`.
@aws-sdk/client-s3 + s3-request-presigner	Durable object storage + short signed URLs for exports (`core/object-io.ts`).
nodemailer 10	SMTP transport only — mail is queued durably in the DB (`EmailOutbox` table) and a claim-then-send worker (`core/mail.ts` + `core/outbox.ts`) delivers it, so a crash can't lose a message.
Google OAuth	Sign in with Google (`features/oauth.ts`).
Google Places (Text Search)	"People/projects nearby" via server-side key in `features/places.ts`, post-filtered by distance, attribution assets included.
OpenAI-compatible endpoint	AI assistant (`core/assistant-provider.ts`, default model `gpt-4o-mini`) behind daily token budgets (`core/ai-budget.ts`, `ai-ledger.ts`).

7 · Shipping & ops

Tool	Use
Docker Compose (`deploy/compose.yml`, root `docker-compose.yml`)	Same stack for local dev (postgres+redis) and production (api, web, db, redis, clamav…). Healthchecks on pg/redis.
Dockerfile	Single image, node:22-bookworm-slim: `npm ci` → `db:generate` + `build:api` + `next build` → runs as the `node` user, ports 3000/3001.
Compose services (`deploy/compose.yml`)	`postgres`, `redis` (appendonly), `clamav` + one-shot `clamav_signatures`, a one-shot **`migrate`** (`npm run db:migrate`) that gates startup, then `api`, `web`, and `proxy` (Caddy).
Caddy	Automatic HTTPS, forwards `/api/*` to the API with rate-limit-safe `X-Forwarded-For`, sets security headers.
systemd (`deploy/projectconnect-monitor.*`)	Periodic ops monitor (quarantine/export/storage alarms).
Backups	There is **no `backup` npm script** — `docs/OPERATIONS.md` prescribes encrypted `pg_dump` + object-store sync, and the repo proves the restore side works via `scripts/verify-local-recovery.mjs` / `verify-local-restore.py` (dump → encrypt → restore into a separate DB → compare → serve). A scheduled off-site backup is still an operator task to provision.
npm scripts	Everything above is driven by `package.json` (see scripts list).

8 · Quality tooling (devDependencies)

Tool	Use
Playwright test 1.64	The entire suite — **118 tests** covering API behaviour end-to-end plus browser tests (auth flows, uploads, storage/quarantine). No Jest/Vitest.
@axe-core/playwright	Automated accessibility scans of key pages in-suite.
ESLint 10 + typescript-eslint 8.71 + react-hooks	Static error catching (`npm run lint`).
Prettier 3.9.9	Formatting consistency (`format:check`).
tsc --noEmit	Whole-repo typecheck.

Tool	Use
Custom verifiers in `scripts/`	`verify-production-smoke.mjs`, `verify-migration-upgrade.py`, `verify-local-recovery.mjs`, `verify-clamd-local.ts`, contract sync, release packager with checksum.

Honest notes (beginners should know these)

- **`jsonwebtoken` 9.0.3 is in `dependencies` but no app code imports it** — `core/oidc.ts` uses `jose` instead. Candidate for removal (unused-dependency audit).
- No CSS framework and no clamd npm wrapper are deliberate: fewer third-party layers on the two most security-sensitive paths (styling & malware scanning).
- Testing the whole product with one runner (Playwright) is unusual but keeps API + browser + a11y checks in a single 1.8-minute suite.
- Redis currently does rate limiting only; the mail queue is DB-backed, so a Redis outage degrades limits, not mail.

Where to look for each claim

`package.json` (versions) · `docker-compose.yml`, `deploy/` (services, Caddy, systemd) · `prisma/schema.prisma` (42 models) · `apps/api/src/core/security.ts` (sessions/CSRF) · `apps/api/src/core/preview-config.ts` (sandbox cookie mode) · `apps/web/next.config.ts` (proxy + CSP) · `.env.example` (full env list: `DATABASE_URL`, `REDIS_URL`, `APP_ORIGINS`, `PUBLIC_APP_URL`, `SMTP_*`, `GOOGLE_*`, `OPENAI_*`, `CLAMD_PORT`, `AI_*` budgets…).

Part III — End-to-End Flow

Beginner flowchart: arrival → registration → login/session → core loop → uploads → security gauntlet → ship.

The same flowchart from today, embedded for reference.

A flowchart for a beginner developer. Top to bottom, each box = one real step in this codebase. **How to read:** follow the ↓ arrows. Grey chips are the **routes/requests**. Files are in the line under each diagram.

Browser = what users see **API** = the back-end that owns all rules

Prisma = type-safe database access **Cookie** = proof-of-who-you-are after login

0 The big picture — who talks to whom

The browser never touches the database. Every request walks this chain:

Browser
React UI, one big `page.tsx` + components

→

Next.js :3000
renders the page, forwards /api
`next.config.ts rewrite`

→

NestJS API :3001
validation · auth · rules · responses

→

Prisma → PostgreSQL
43 tables, all data lives here
`schema.prisma`

Redis
rate limiting (anti-spam counters)

Mail queue + SMTP
activation & notification emails

ClamAV
virus scan for every upload

Google
OAuth sign-in · Places “nearby” (server-side key)

1 A visitor arrives (logged out)

GET / — page renders on the server

Next.js runs `apps/web/app/page.tsx`

↓

GET /api/projects

public list of project cards → works for everyone

GET /api/me

no cookie →

401

… which is NORMAL. UI flips to “guest mode” with a Sign-in button

Back-end: `apps/api/src/main.ts` · `@Get("projects")` `@Get("me")`

2 Creating an account

Register form submitted

`POST /api/auth/register`

↓

shape check

zod validation + bcrypt password hash

the plain password is never stored

↓

User row saved — status INACTIVE

an attacker can’t skip email by forging the status field: input DTOs don’t include it

↓

queue → SMTP

Activation email with one-time token

`POST /api/account/verify`

↓

link clicked

Active → sign in

dev data: `prisma/seed.ts` creates sample profiles (blocked in production) — the one-click demo login was removed 18 Oct

Files: `apps/api/src/main.ts` · `apps/api/src/core/security.ts` · `apps/api/src/core/mail.ts` · `prisma/seed.ts`

3 Login → the session cookie (heart of the app)

POST /api/auth/login

browser sends email + password + its Origin header

↓

gauntlet 1

Exact-origin CSRF check

Origin must match APP_ORIGINS (sandbox preview URL is allow-listed in dev)

↓

gauntlet 2–3

Rate limit (Redis) → bcrypt compare → session row (hashed token)

↓

Set-Cookie: pc_session

HttpOnly · path=/ · Secure in production · SameSite=Lax · (preview embed = None + Partitioned)

↓

browser attaches it automatically, forever after

Every later request → guard reads cookie → knows you

↓

403 ORIGIN_FORBIDDEN / 401

bad origin or bad credentials → generic message, no hints about which one

Real bug, real lesson: the demo “not working” in the preview was *this first box* — the preview’s HTTPS domain wasn’t allowed, so the login POST was refused. Debug flow: open DevTools → Network → look at the status code → match it to this diagram.

Files: `apps/api/src/core/security.ts` · `apps/api/src/core/preview-config.ts` · `apps/api/src/core/rate-limit.ts` · Google route: `apps/api/src/features/oauth.ts` (start → Google → callback → same cookie)

4 The core product loop — find a team, join a team

Browse · search · filter cards

categories, skills, location — UI state in `page.tsx`

↓

Open a project

`GET /api/projects/:id`

↓

Apply with a message + what you bring

`POST /api/projects/:id/apply`

↓

Owner reviews applications

`GET /api/applications` `PATCH /api/applications/:id`

↓

accept

Both sides become connections → chat opens

`GET+POST /api/messages/:id`

↓

…then the extras orbit: bookmarks, feed, invitations, team, workspace files, GitHub links

same pattern: component → /api route → guard → Prisma

Files: `apps/api/src/main.ts` · `apps/api/src/features/social.ts` · `apps/web/app/components/Invitations.tsx` , `Workspace.tsx`

5 Uploading an image (avatar / project photo)

The most mysterious feature for beginners — here is every gate it passes:

Pick file in the upload box

`ImageUpload.tsx` checks type + size, sends immediately

`POST /api/account/avatar (or projects/…/images)`

↓

server never trusts the client

Magic-bytes recheck + real image dimensions

a .jpg renamed from .exe dies here

↓

Temp file → ClamAV scan

persistently queued so a crash can't skip it

↓

Infected / failed → 400

never stored, never served

Clean → durable storage + SHA-256 digest

DB links it: `avatarId` / project imageId

↓

Client refetches → gets the server URL

`GET /api/images/:id (audience-checked)`

↓

`PreviewImage` **renders: loading · OK · failed + Retry**

"still scanning" or "quarantined" shows a clean message, never a broken-image icon

Beginner insight: the thumbnail you picked lives only in your RAM. What you see *after saving* always comes back from the server. So a "preview not working" report means: check upload response, then `/api/images/:id` status, then cookie/origin — in that order.

Files: `apps/web/app/components/ImageUpload.tsx`, `PreviewImage.tsx` · `apps/api/src/features/images.ts` · `apps/api/src/core/file-scanner.ts`, `storage-scan.ts`, `public-image.ts`

6 Every write request runs the same gauntlet

CSP + security headers
next.config.ts (frame-ancestors, img-src, …)

→

Rate limit
core/rate-limit.ts + Redis

→

Exact-origin CSRF
core/security.ts

→

Session cookie → user

→

zod validation

→

Prisma (parameterized)

→

DTO response
never leaks password hashes or tokens

7 Run it & ship it

1 · Install & services

`npm install` · Docker Postgres + Redis

↓

2 · Database shape

`prisma migrate` (25 numbered migrations) · dev: `npm run db:seed`

↓

3 · Run both servers

`npm run start:api` :3001 → `npm run start:web` :3000 (web needs INTERNAL_API_URL)

↓

4 · Prove it works

`npm test` (118) · build · lint · `scripts/verify-production-smoke.mjs`

↓

5 · Staging env

DATABASE_URL · APP_ORIGINS=https://yourdomain · SESSION/CSRF secrets · SMTP · GOOGLE_CLIENT/SECRET/PLACES · CLAMD · AI key · PUBLIC_APP_URL

↓

6 · Ops

backup procedure per `docs/OPERATIONS.md` (encrypted pg_dump + object store; restore rehearsed by `verify-local-recovery.mjs`) · health /api/ready · systemd monitor timer

↓

7 · Launch checklist

`LAUNCH_HANDOFF.md` — domain, HTTPS, CSP, moderation rota, support desk

Five words that unlock everything: API (rule-owning back end) · **cookie** (proof you're logged in) · **ORM** (typesafe DB access) · **migration** (replayable DB change) · **CSRF origin check** (reject writes from unapproved sites).

Diagrams also available as Mermaid in `docs/BEGINNER_FLOWCHART.md` (paste into GitHub/Notion to render).

Part IV — For Users

What the website does for a member, screen by screen.

What the website does for you, screen by screen. Written so a first-time builder can follow every step.

1 · Getting in

1. **Open the site.** The home page (Discover) shows real project cards even before you sign in.
2. **Create an account** → Sign in → Register. Give a name, email and a strong password. Check your inbox and click the activation link (one-time, expires, and nobody can tell whether an email is registered — spam harvesters get nothing).
3. **Sign in** two ways: email+password or **Continue with Google**.
4. **Session basics:** staying signed in is a single HttpOnly cookie. Account security → Sessions lists every device; revoke any one, or **log out everywhere** (this also happens automatically when you change your password).
5. Forgot the password? Forgot password → one-use emailed link → new password. Old links die after first use or expiry.

2 · Be findable: your profile

- **Edit your profile** (top-right avatar button): bio, education, portfolio URL, city, **skills** and **interests** — skills power project matching and what owners see when you apply.
- **Profile photo:** upload any PNG/JPEG. The file is virus-scanned and security-checked first, so the avatar may briefly show “loading” or a Retry preview button — that's the safety pipeline, not a bug. Delete it any time.
- **What's public:** name, initials/avatar, bio, skills, interests, city, portfolio link, your public projects. **Private forever:** email, password, sessions, tokens, your draft applications' contact details.

3 · Find your people (Discover, People, Nearby)

- **Discover:** search box + category chips (drones, renewable energy, agriculture, hardware…) + skill filters. Cards show needed roles, capacity and tags.
- **Saved projects:** bookmark anything from a card; find the list later in the sidebar.
- **People:** browse makers, follow them, or send a **connection request** (they accept/decline; either of you can end it; **block** removes visibility and messaging both ways). Match scores are shown with the why (shared skills, city, activity) — never a mystery percentage.
- **Nearby hardware:** finds hobbyist/electronics stores around a city you type — distance-sorted, with hours/phone when available, directions links, and your favorites. Community members can submit stores and flag bad data.
- **Activity feed:** the public pulse — new projects, team wins, posts you can like. Post your own updates; delete them any time.

4 · Launch or join a project (the core loop)

Joining: open a card → read roles, skills, timeline → **Apply** with a short pitch and what you bring → the owner reviews in My projects → Applications → accept → you're on the team: chat opens, you appear in the workspace, notifications tell both sides. **Launching:** My projects → New project: title, description, category, cover image, skills wanted, **open positions with capacity** (e.g. “2 firmware engineers” —

enforced atomically, so the 3rd accept can't oversubscribe), location/remote, status. Edit or archive later; deleting a project cleans up its images and applications truthfully. **Managing applications:** owners accept/decline with a note; applicants may cancel or reapply under the project rules; capacity, duplicate-acceptance protection and audit events are automatic.

5 · Work together (the Workspace tab on your project)

Tab	What it is for
Overview	What's due, who's active, pinned status
Tasks	Kanban board — drag with a mouse or move with the keyboard; priority, assignee, due date, labels
Milestones	Group tasks; progress % is computed from real task states, not vibes
Team	Invite by link/username, change roles, remove members, **transfer ownership**, leave gracefully
Files	Private project files (quota'd, scanned, deleted-or-captured automatically) — only members can read them
GitHub	Link repos; stars/language/last-push metadata cached; connect a private token only if you choose to
Updates	Shipped progress posts that flow into the Activity feed
Invitations	Pending team invites in one place
Chat	Per-project channel + direct messages: unread badges, gap-safe catch-up after a dropped connection, read receipts, and retry — no silent message loss, and blocked users cannot reach you

6 · Level up (Resources & AI assistant)

- **Resources hub:** community-curated repos, docs, APIs, datasets, tutorials. Search/filter, upvote, bookmark, and **attach to your project** so teammates see the same toolbelt. You can submit resources; moderation verifies them.
- **AI assistant:** asks about your project → ideas, stack picks, debug hints, roadmap drafts. Guardrails you should know: per-day cost caps (yours and global), your chats are private to you, and every external citation is labelled verified or unverified — **it will not invent GitHub repos or shops**, and drone/electrical answers carry safety disclaimers. Turn “roadmap → tasks” into real board items with one confirmation (duplicate-safe).
- When an integration isn't configured by the site operator yet, you'll see an honest not configured note — never a fake success screen.

7 · Your account, your data (Account security view + /account)

- **Preferences:** which notifications you get (mentions, applications, messages…).
- **Sessions:** every active login with device/time; revoke individually or all.
- **Export my data:** queues a ZIP (profile, projects, messages, files list…) — you get a download link when ready; large exports keep working even if a server restarts mid-zip.
- **Delete account:** after confirmation, your profile, applications, messages and uploads are removed or anonymised per the retention rules; files still needed as legal evidence/quarantine stay sealed and invisible — that's intentional.
- **Reports:** flag a profile or project → the moderation queue → actions taken feed back into your notifications.

8 · Small things that matter

- **401 in the console on first load** = you're simply logged out; the site uses it to show guest mode. Normal.

- Images from the preview environment may need cookies; if a photo won't load inside an embedded frame, open the preview in a new tab.
- Rate limits protect logins, uploads and messages — “slow down” responses are self-healing; wait the shown seconds.
- Works from 320px phones to ultrawide; full keyboard + screen-reader-friendly dialogs (focus is trapped and restored, Esc closes); reduced-motion honoured.

The one-sentence rule of this site: everything you post is yours, everything you share is consented, and every team decision the platform makes (capacity, acceptance, access, scan results) is enforced by the server — not just drawn by the UI.

Part V — For the Owner

Setup, admin, operations, money, and the honest launch state.

What running this website means for you: setup, administration, operations, money, risk, and the honest launch state.

---

0 · Your two roles, separated on purpose

- **Owner/company (you):** buys hosting, holds credentials, approves legal & AI spend, authorises publication. Most of §1–§3 is yours.
- **Admin operator (staff):** runs `AdminPanel` on the site — moderation queue, suspensions, telemetry. That part is software you already have; nobody needs a terminal for day-to-day community care. The engineering deliberately cannot fake your half: no deployment, purchase, DNS, or policy approval exists until you do it.

1 · Before users arrive (one-time setup — `docs/MANUAL_SETUP.md` has exact steps)

1. **Hosting + domain + DNS** — a Linux host (or two containers) with TLS. Deploy stack: `deploy/compose.yml` = postgres 17, redis 7, clamav + signature updater, one-shot `migrate`, `api` (:3001), `web` (:3000), **Caddy** (`deploy/Caddyfile`) for automatic HTTPS, HSTS and `/api` proxying with rate-limit-safe client IPs.
2. **Environment** (from `.env.example`): `DATABASE_URL`, `REDIS_URL`, exact `APP_ORIGINS=https://yourdomain`, `PUBLIC_APP_URL`, `SMTP_URL` + `MAIL_FROM`, `GOOGLE_CLIENT_ID/SECRET`, `GOOGLE_PLACES_*`, `OPENAI_API_KEY` + `OPENAI_MODEL` +price/budget keys, `CLAMD_PORT`. Secrets stay in the host secret store — never in git or the release archive.
3. **Google Cloud:** consent screen + authorised redirect URIs (`https://yourdomain/api/auth/google/callback`); enable Places API with **billing alerts and caps**; keep the server key restricted to Places and never ship it to browsers.
4. **Email provider:** verify your sending domain (SPF/DKIM). Activation and reset mail are business-critical; the DB outbox retries so provider blips don't lose mail.
5. **Object storage (optional but recommended):** S3-compatible bucket + credentials for durable files/exports; short signed URLs only; private volume otherwise.
6. **AI budget governance:** you pick the approved model, **unit price (per-million tokens)** and daily ceilings (`AI_*` keys). The ledger enforces integer micro-USD per user/project/global day — the platform will not overspend your chosen numbers, but it can't approve them for you.
7. **GitHub App / OAuth** (only if you want richer repo previews): register app, webhook secret server-side.
8. **Legal:** privacy policy / ToS / retention drafts exist in-repo marked draft — get counsel approval before launch.

2 · Launch gates you are responsible for (and the honest current state)

- The codebase stands at **NOT READY FOR PRODUCTION LAUNCH** (`docs/LAUNCH_STATUS.md`): 118 automated tests green, migrations/restore/scanner drills locally verified, full feature set implemented — **but** the remaining engineering (patched scanner deployment, live-S3 semantics, representative load & multi-replica proofs, independent security review) and **all** external executions (real Docker host, hosted CI, edge, alert delivery, off-site RPO/RTO) stay open.
- Do not read the local demo, the preview URL, or an archive as a deployment. Launch = your authorization **plus** the production-mode smoke test passing behind Caddy with real providers.

- What we never do for you: create accounts, spend money, register domains/keys, or approve policies. The agent-side list in LAUNCH_STATUS tells you what's left before that gate.

3 · Day-to-day operations (what your staff actually touches)

Need	Tool	How
Moderate reports, take down content, suspend/restore users	**AdminPanel view** on-site	Least-privilege: admin role checked per mutation; every action audit-logged with reason
Watch the system	`npm run monitor` + systemd templates (`deploy/projectconnect-monitor.*`)	Alerts on queue depth, scan backlog, quarantine count, export failures, disk — aggregates only, no private metadata
Health check / uptime probe	`GET /api/health` , `GET /api/ready`	ready verifies DB+Redis actually answer
Storage duty	`scripts/storage-rescan.ts` (dry-run by default), `scripts/storage-inventory.ts`	Re-scan on demand; orphans aged & quarantined through the same safe pipeline
Malware engine freshness	`clamav` service signature updater (compose)	Keep the updater container; treat "engine outdated" warnings as alerts, not noise
Backups	**Procedure** in `docs/OPERATIONS.md` : encrypted `pg_dump` + object-store sync, scheduled (cron/systemd timer)	The repo guarantees the restore side is proven: `scripts/verify-local-recovery.mjs` / `verify-local-restore.py` do dump→encrypt→restore into a fresh DB→compare all 43 tables→serve restored API. Schedule it off-site yourself
Incident response	`docs/SECURITY.md` + threat model	Rotate: session signing key ⇒ global logout; SMTP/OAuth/AI keys instantly; revoke a user's sessions from AdminPanel; quarantine hides bytes without deleting evidence
User data requests (export/delete)	Automated in-app (§7 of USER_GUIDE)	Exports survive restarts; deletion honours legal-hold/scan-dependency pins — that's compliance, not a bug

4 · What the platform enforces so you don't have to babysit

- **Security on every write:** exact-origin CSRF, HttpOnly cookie sessions, rate limits via Redis, CSP + HSTS + nosniff headers, zod schema validation, parameterised Prisma queries, DTO serializers (passwords/tokens can't leak, even nested).
- **Upload safety:** client check → server magic-byte + real-dimension verification → ClamAV scan queue → sha256 + durable rename that can never overwrite → audience-checked serving → quarantine path with evidence preserved. Users see honest “scanning/failed/retry” states.
- **Fair-play rules as data, not UI:** project capacity, one-accept-per-application, connection uniqueness, blocked-user messaging denial, one owner per project — all DB-constrained + transaction-tested.
- **Cost ceilings:** AI daily budgets per user/project/global; Places per-user/day query caps; export jobs capped (3/user/day, size + object limits) — billing surprises are structurally limited.
- **Abuse surface shrinking by default:** enumeration-neutral auth errors, signed short-lived pagination cursors, one-use expiring tokens, no executable content served on your origin, SSRF-blocked server fetches.

5 · Money and meters you should watch (weekly, 10 minutes)

1. **AI spend** — ledger rows are priced at the unit price you approved; alert if ceilings are hit routinely (feature-market signal, or budget too tight).
2. **Places API** — billing dashboard vs your caps; disable browser Maps display entirely if you never enable it (the current design needs no public Maps JS key).
3. **SMTP** — bounce/complaint rates; the outbox retries, mailbox providers punish volume spikes.
4. **Object storage + DB size** — `storage:inventory` dry-run reports; growth should track users, not incidents.
5. **Uptime/monitor alerts** — if the systemd monitor emails nothing for weeks, test its alert path once a quarter.

6 · Upgrade & change discipline (for whoever maintains the code)

- Migrations are forward-only, numbered, checksum-tested **and** upgrade-tested from older releases (`verify-migration-upgrade.py` pattern). Run the `migrate` one-shot before `api` starts (compose enforces).
- Backend builds before restart (`npm run build:api` → `npm run start:api`); the web needs `INTERNAL_API_URL` at **build time**.
- Release flow: `npm run release` → sanitized archive + sha256 (root `ProjectConnect_release.*` mirrors the workspace) → deploy from the same commit you tested; never seed production (`db:seed` is dev-guarded).
- `docs/VALIDATION_REPORT.md` is the source of truth for "what was actually run"; treat any claim not in it as unverified, including claims from this guide.

7 · Where every answer lives

Question	File
What did we build & why	`docs/MASTER_PROMPT.md`
Which tech, which versions	`docs/TECH_STACK.md`
How each flow moves through the system	`docs/BEGINNER_FLOWCHART.md` (+ `.html`)
What users can do	`docs/USER_GUIDE.md`
This file (owner ops)	`docs/OWNER_GUIDE.md`
Exact setup you must do manually	`docs/MANUAL_SETUP.md`
Runbook: backup/rescan/monitor details	`docs/OPERATIONS.md`
Security model & response	`docs/SECURITY.md`, `docs/THREAT_MODEL.md`
Integration contracts (Google/GitHub/AI/S3)	`docs/INTEGRATIONS.md`, `docs/API.md`, `docs/openapi.json`
Current honest status + remaining gates	`docs/LAUNCH_STATUS.md`, `docs/VALIDATION_REPORT.md`
The human entry point	`LAUNCH_HANDOFF.md` (repo root)

Part VI — Revenue Streams

How this platform monetises, mapped to the enforcement points already in the code.

How this specific platform makes money, mapped to the enforcement points that already exist in the code.

(All prices/ranges are planning assumptions, not validated market data — they exist so the owner can decide. Anything marked **needs-build** requires new engineering.)

Why the map works in our favour

Monetisation usually fails when it needs brand-new trust machinery. ProjectConnect already enforces **per-user daily AI budgets in integer micro-USD (`AI_USER_DAILY_MICROUSD_BUDGET`), quotas, roles, capacity rules, invites, moderation, export/delete, and a DB-backed mail queue**. That means most paid tiers are raising existing gates, not inventing new ones — the honest, cheap way to price.

The streams

1 · Builder Pro (consumer subscription) — the core line

Free: everything a member needs to join and ship (browsing, applying, chat, workspace, 1 project, base AI allowance). Pro (suggest **$7/mo · $60/yr · students free via verified campus email**):

Pro benefit	Where it plugs in	Build cost
Bigger daily AI budget + priority model routing	AI ledger ceilings are per-user integers — already multi-tier capable	small (a `plan` column + ceiling lookup)
Advanced filters & explainable match deep-links (search "firmware + battery + within 50km + open to students")	discovery/search is server-side with pagination	small
Showcase boost (pinned in category for 7 days, clearly labelled Promoted)	cards already render from API; needs a slot field	small + moderation rule: always labelled
Unlimited active owned projects (free = 1…2)	owner-capacity checks already exist	trivial
Private "co-founder match" reports	fit-v2 scoring exists	medium
Needs-build: billing (Stripe/Paddle merchant-of-record for global VAT), plan gate middleware, refund/export interplay (billing data lives at the processor — disclose it in the privacy draft)	—	medium

2 · Teams & Club plans (seat billing)

Student clubs, maker spaces, small hardware startups need: more workspace members than free allows, larger private-file quotas (S3 quota code exists), milestone/velocity summaries across projects (task data is persisted — reporting is a read-model), GitHub integration per team (already built). Suggested **$25/mo per 10 seats**; club verification through the existing admin flow. Seat caps enforce themselves via the team-role system.

3 · Hiring & talent (B2B, highest ceiling)

The product is a portfolio of people who shipped real things — recruiters pay for that signal. - Job/partner post tied to open positions (positions with capacity already exist — a "hiring" position type is a label + queue). - Sponsor-a-project: a company funds an open-source hardware project's parts/AI budget in exchange for a labelled sponsor badge. - **Never** sell raw user data or scraping — profiles are user-owned; recruiter views show only public fields, exactly the serializer that already exists.

4 · Local hardware partnerships (the moat stream)

The Nearby page lists our own community store DB (Google Places results are transient by policy — so ads go on our records, which is legally clean). - Stores claim listings free → verified badge, hours, offers, “sponsored” slot at top of a city (clearly labelled). - Coupon/partnership campaigns for hackathon parts packs. Suggested **$19–49/mo per store** + revenue-share on tracked redemptions (needs-build: simple redemption codes).

5 · Events, hackathons, content

Ticketed virtual hackathons (platform already has teams, invitations, milestones, chat channels — that is a hackathon product), sponsor packages, and a “Build Log” editorial arm (the feed + Resources hub double as CMS). Mostly operations cost, small engineering.

6 · White-label for universities/incubators (needs-build, later)

Campus-branded instance per tenant. Honest status: current schema is single-tenant; a `campus` scoping column + subdomain routing is the minimum. Price **$2–10k/yr per campus**. Only pursue after #1 proves out.

7 · Marketplace / commerce cut — explicitly deferred

Parts resale or group-buying wants escrow, payments, dispute tooling: real engineering + legal exposure. Parked until ≥25k MAU; do not promise it early.

Unit economics sanity (assumptions, re-measure monthly)

- AI: at the repo's own approved price ceiling, a Pro user's daily ceiling × 30 days is the worst case monthly COGS — cap Pro tiers so AI-COGS ≤ ~25% of $7; the ledger already reports spend per user (admin telemetry).
- Hosting at launch scale is one small VPS + managed DB ≈ $35–70/mo → break-even at ~10 Pro subs.
- Store/hiring streams are near-zero marginal cost — they fund discounts for students.

Sequencing (do not parallelise money experiments)

1. Free growth → 2. Builder Pro → 3. Club seats → 4. Store partnerships → 5. Hiring → 6. Events → 7. White-label → 8. Commerce.

Rules that protect the brand (already true of this product; keep)

Every paid placement labelled Promoted/Sponsored; free-forever students; no dark patterns in cancellation (export & delete are first-class features); invoices/PII stay in the processor, app export discloses that. Revenue decisions are owner decisions — the engineering only enforces them.

Part VII — Launch, Deploy & Hosting

Six launch phases, the recommended deployment method, step-by-step runbook, scaling ladder, go/no-go gate.

Step-by-step ways to launch, and the recommended deployment method. Commands match this repo (compose v2, `deploy/`).

A · The launch path — six phases, no skipping

Phase 0 · Code-ready (agent territory). All gates in `docs/LAUNCH_STATUS.md` close: suite green, builds pass, security items verified. Current honest state: 118/118 locally, but several production-proof gates (live S3, patched scanner deployment, load, off-site RPO/RTO, hosted CI) are still open → **not launch-ready yet**.

Phase 1 · Owner prerequisites (only you). Domain + DNS, host/billing, SMTP domain verification, Google OAuth consent + Places enablement, LLM key + approved price, S3 bucket, policy approval. `docs/MANUAL_SETUP.md` is the checklist.

Phase 2 · Staging dress rehearsal. Deploy the identical production stack to a staging host (`staging.yourdomain`). Run `node scripts/verify-production-smoke.mjs` against it, execute one full **backup→encrypted restore drill** (the 43-table drill is already automated: `scripts/verify-local-recovery.mjs`), run migrations on a copy of real-shaped data. Nothing counts as rehearsed unless restore was executed.

Phase 3 · Closed beta (product supports this natively). In production the app **forces registration privacy** (`PRIVATE_REGISTRATION` cannot be off in prod): signups become “Pending account” with email-activated onboarding — the codebase already implements invite-style gating. Seed 50–200 real builders via club partners; watch `/api/ready`, admin telemetry, moderation queue. Exit criteria: 2 weeks, ≥35% of accounts complete a profile, zero unresolved Sev-1.

Phase 4 · Graduated launch. Open signups per city/cohort (e.g., 3 universities at a time) so rate limits, Places quotas, and AI budgets scale predictably. Keep the waitlist as a throttle valve.

Phase 5 · General availability + ritual. Weekly: check monitor alerts, signature freshness, backup success log. Monthly: restore drill for real, patch images (`npm audit`, ClamAV `stable` tag, Prisma patch bumps with tests). Quarterly: security review against `docs/THREAT_MODEL.md`.

B · Best deployment & hosting method (recommended = “Boring Strong”)

Recommended: one small VPS + Docker Compose + Caddy. `deploy/compose.yml` already defines postgres 17, redis 7 (AOF on), `clamav/clamav:stable` + a one-shot signature service, a **one-shot `migrate`** that runs before the apps (migrations-before-rollout is enforced), `api`, `web`, and Caddy bound to 80/443 only; app services stay on a private network. This is genuinely the right size for ≤ ~10–20k MAU.

Host type	When
VPS (Hetzner/DO/Lightsail, 4 vCPU / 8 GB ≈ $25–65/mo)	Default. All-in, full control, cheap.
Managed PaaS (Fly.io/Railway/Render) for web+api, managed Postgres, self-run worker for scans/queue	When you don't want to own OS patches. Cost 2–3×.
Kubernetes	>50k MAU or multi-service teams. Not now.

Splits worth doing from day 1 even on a VPS: managed/replicated Postgres or **WAL-G backups to S3** (DB is the business), object storage for files (already supported), external uptime probe (health checks exist).

Honest anti-recommendations: don't deploy straight onto a raw cloud VM without compose (drift, “secrets in home bash”), don't use Firebase/Supabase auth as a bolt-on (sessions/CSRF design already

exists and is tested), don't add CDN in front of HTML before CSP is verified end-to-end (it will break `frame-ancestors`/OAuth callbacks — add CDN for `/_next/static/*` assets only, later).

Step-by-step deploy (first time, VPS)

1. Provision Ubuntu 24.04 LTS, 4 vCPU/8 GB, in the region near your users. `adduser deploy`, key-only SSH, `ufw allow 22,80,443`, enable `unattended-upgrades`.
2. Install Docker Engine + compose plugin. Install `restic` (or `rclone`) for off-site backups.
3. DNS: `app.yourdomain → A record`, `www` redirect. Caddy (`{$APP_DOMAIN}` in `deploy/Caddyfile`) gets TLS automatically on first request — HSTS, nosniff, `X-Frame-Options DENY`, `/api` reverse-proxy to `api:3001` with client-IP pinning already configured.
4. Put the real `.env` in `/etc/projectconnect/.env` (600 perms). Required: `POSTGRES_PASSWORD`, `DATABASE_URL`, `APP_ORIGINS=https://app.yourdomain`, `PUBLIC_APP_URL` same, `REDIS_URL`, SMTP, Google OAuth + `GOOGLE_PLACES_*`, OpenAI key/model/price/budgets, `CLAMD_HOST/PORT`, backup/S3 vars.
5. Build the app image from the repo's Dockerfile (node:22, single image, `npm ci` → `db:generate` → `build:api` → `next build` with `INTERNAL_API_URL=http://api:3001`). In compose, point `api`/`web` at this image.
6. `docker compose -f deploy/compose.yml up -d` → one-shot `clamav_signatures` → `migrate` (25 migrations) → postgres/redis/api/web/caddy.
7. `curl https://app.yourdomain/api/ready` green; run `node scripts/verify-production-smoke.mjs`; run the login/register/payment-free smoke list.
8. Enable the monitor: copy `deploy/projectconnect-monitor.{service,timer}` (systemd), start the timer — queue/quarantine/export alarms arrive without reading logs.
9. Backups: nightly `pg_dump | encrypt → S3/off-site` + file-store sync (documented procedure in `docs/OPERATIONS.md`; restore side is what the repo's drill proves). Restore is the thing you schedule and test, not the dump.
10. Email reputation: DMARC/SPF/DKIM on your domain before beta invites — activation mail that lands in spam kills launch metrics.
11. Freeze a release tag; deploys from then on = `git pull` → `docker compose build api web` → `compose up -d` (the `migrate` one-shot runs first by `depends_on`). Zero-downtime is not required at this scale; a 20-second blip at 3 a.m. is the honest trade.

Scaling ladder (when, and only when)

1k MAU: nothing changes → 5k: move Postgres to managed/read-replica-ready, add S3 for files (flag already exists) → 20k: split api onto a second VPS behind Caddy; sessions are DB-backed and rate limits Redis-backed, so **no sticky sessions needed** → 50k+: queue workers and scans on their own box; only then consider k8s.

Rollback rule

Migrations are forward-only, so rollback = redeploy **previous image** (schema stays compatible because changes are expand-then-contract; the repo's upgrade tests 20→25 and 22→25 prove the pattern). Keep the last 3 images tagged.

C · Go/No-Go gate — all must be true before public launch

☐ LAUNCH_STATUS gates closed or explicitly accepted as risk (with sign-off) ☐ staging smoke passed ☐ restore drill executed on staging ☐ email domain verified + deliverability checked ☐ legal docs approved ☐ owner approved AI price/ceiling + Places budget ☐ monitoring timer active + one test alert delivered ☐ backup cron ran once off-site ☐ rollback rehearsed once. **Anything unchecked = not launch-ready — no exceptions for deadlines.**

Part VIII — Sales & Marketing

Positioning, buyers, 90-day launch motion, channels, sales plays, metrics, budget.

Built from what this product actually does, so every promise is keepable. Targets are planning assumptions to measure against, not claims.

1 · Positioning

One line: Where hardware ideas get built — find teammates, ship the project, keep the receipts. **Enemy (real, non-abstract):** Discord servers where projects die, LinkedIn where makers perform, hackathons where teams disband at 6 a.m. ProjectConnect is the boring glue: teams, tasks, files, chat, resources, local parts — in one place that persists. **Proof lines you may use (all enforced in code):** server-enforced team capacity · private scanned file storage · per-day AI cost ceilings (no runaway bills) · local hardware directory with real distance math · account export/delete in two clicks · “NOT configured” honesty instead of fake features.

2 · Who buys, who uses (and they're different people)

Segment	Value	Where to find them
Student builders (primary)	Free; teammates + mentorship + AI planning	Robotics/EDA/42-coder clubs, maker faires, university societies (Chennai/Hangzhou/Berlin-type clusters beat national campaigns)
Independent hardware devs	Hiring + collaborators + store deals	X/Bluesky build-log posters, r/hardware, Hackaday comments, Tindie/Etsy maker lists
University clubs/labs (buyer #1)	Club plan = organisation without spreadsheets	Direct email to faculty sponsors & club presidents, one campus at a time
Hardware stores / suppliers (buyer #2)	Sponsored local listings, community demand signal	Walk in. Seriously. Local electronics markets are the entire moat
Hiring companies (buyer #3, later)	Portfolio-verified candidates	Founder LinkedIn outreach + 2 partner universities first

3 · The launch motion (90 days, sequenced)

Weeks 0–4 · Pre-launch — supply before demand. An empty network is the death of this category. Recruit **30 anchor teams** by hand (club projects already in flight get free Club plan + “founding” badge — costs $0). Seed 60 stores into the Nearby directory via friendly shop visits (their listings + optional offers). Build-in-public: one weekly post from the founder showing real commits, metrics, and bug autopsies — this audience rewards honesty; our whole codebase is built on it. **Weeks 5–8 · Closed beta — the product literally supports it** (production registration = pending + email activation). Waitlist page → admit 25/week from referrals first (invite code loop exists in-app). KPI: **activation = first connection made within 14 days**, not signups. **Weeks 9–12 · Public wave — one region.** Pick one city cluster (e.g., South Indian engineering colleges OR Shenzhen hobbyist scene) and win it: campus hackathon sponsorship (~$200–500), one “find-your-cofounder” night per university, flyers with QR to their own project page (per-project pages + sitemap are already SEO-ready — real product capability). Then let case studies, not ads, widen.

4 · Channel-by-channel playbook (ranked by expected yield for this product)

1. **Club partnerships (top)**: one signed campus club = 40–150 organic users with instant teams. Offer: free club plan, founding badge, our hackathon sponsorship kit.

2. **Build-log content engine**: teams posting progress is the product loop (activity feed exists). Externalise: weekly best-builds email/X thread; ask every activated team at day 21 for their log.
3. **SEO — project pages, not blog spam**: shareable stable URLs + metadata + partitioned sitemaps are implemented; long-tail queries ("drone ESC tuning team project") are exactly what indexed project cards answer. Expect months, not weeks — seed content must be real projects, never placeholders.
4. **Local stores as co-marketers**: shop owners tell everyone who walks in; a "Find parts + teammates here" window sticker costs nothing and maps 1:1 to the Nearby feature. Sponsored listings sold face-to-face at launch.
5. **Reddit/HN carefully**: maker subreddits reward a genuine "we built an open-hardware team platform, here's the failure log" post; show-HN with the demo dataset. One well-timed post ≈ 1–3 weeks of waitlist. Never astroturf — this audience executes your startup by reading your code comments.
6. **Paid (only after activation ≥35%)**: tiny X/Reddit tests at $10/day targeted at hardware interests, creative = real project cards; kill rule: CAC > $4 per activated user for a free product = stop.
7. **Events**: sponsor a drone/robotics track at 2 college fests per term; the prize = Club plan + AI credits. Cheap, on-brand, produces content.

5 · Sales plays for the three buyers

- **Clubs (self-serve-ish):** faculty email: 90-word offer (free founding club) → 20-min demo → semester MoU. Renewal trigger = end of term; owner: student ID verification for free Pro tier is a **needs-build** item (don't promise until coded).
- **Stores (founder-led, local):** walk-in pitch with live demo of their own listing; free first 30 days; $19–49/mo sponsored tier (see REVENUE_STREAMS §4). Upsell: post their stock arrivals as feed items (feed already renders rich text/links; no scraping of their data — they post it, unlike transient Google Places content which the product deliberately never stores — keep it that way).
- **Hiring (post-scale):** two design partners free for 6 months in exchange for case studies; price only when 200+ public "open to work" profiles exist. Never sell raw data — recruiter views use the same public serializer as everything else (already enforced).

6 · Funnel metrics & rituals (instrumented with what exists)

Funnel: visit → signup → pending-activated → profile complete → first apply → first connection → 14-day return → first shipped update (feed post = retention gold). Admin telemetry + audit + outbox delivery logs give most of this **today**; a metrics page is **needs-build**. Weekly ritual (30 min): activation %, 14-day return %, store-listing growth, AI-budget burn vs ceiling (ledger), moderation queue age. Every public number gets the same discipline as code: measured or marked estimate.

7 · Budget (three tiers — start at first)

Tier	$/mo	Buys
Bootstrap	0–50	Founder time, stickers, hackathon swag
Regional push	150–400	1 event sponsorship + tiny paid tests + email tool
Post-PMF	1k–3k	Content freelancer, 2 events/term, referral merch

8 · What we explicitly don't do (keeps brand and codebase aligned)

No bought followers/users (empty networks are visible in a network app), no spam cold blasts without consent (the mail queue is transactional by design), no scraping competitors' or Google's restricted

content for marketing, no “AI does everything” claims (assistant ships with honest budget/consent/citation states), no launch countdown theatrics while a Go/No-Go gate is open. **Marketing promises must be verifiable in the LAUNCH_STATUS matrix — same evidence rule the engineering follows.**